

SMARTGEAR RETAIL 2.0

Data Engineering Capstone Report

Assignment 2.0 — Modern Lakehouse & Streaming Platform

Kafka • Databricks • Delta Lake • Medallion Architecture • Vector Search

Technologies Used

Apache Kafka (Confluent Cloud) | Databricks Community Edition | PySpark
Delta Lake | Spark Structured Streaming | ChromaDB | Sentence Transformers

Submitted by: Rishabh Srivastava
Postgraduate Program in Data Science, Data Engineering and AI: Batch 21

Table of Contents

- 1. Project Overview 3
- 2. End-to-End Architecture Design 4
- 3. Kafka Pipeline & Streaming 6
- 4. Medallion Architecture — Bronze / Silver / Gold 9
- 5. Gold Layer KPIs & Advanced Analytics 13
- 6. Semantic Search & Vector Database 17
- 7. Orchestration & Pipeline Design 19
- 8. Monitoring & Incident Response 21
- 9. Learnings & Reflections 23

1. PROJECT OVERVIEW

1.1 Business Objective

SmartGear Retail 2.0 is a modern retail analytics data platform built to simulate a real-world enterprise environment. The project combines batch processing of historical sales data with real-time streaming via Kafka, enabling scalable analytics and business intelligence across multiple retail dimensions.

1.2 Business Goals

- Analyze retail sales performance across stores and regions
- Monitor real-time regional revenue trends
- Identify top-performing products and stores
- Enable semantic product search via vector embeddings
- Build a scalable, enterprise-grade Lakehouse architecture

1.3 Dataset Strategy

Dataset	Description
Batch Dataset	smartgear_sales.csv — historical transaction records from Assignment 1
Streaming Dataset	
Product Metadata	

1.4 Technology Stack

Layer	Technology	Purpose	Platform
Ingestion	Apache Kafka	Real-time event streaming	Confluent Cloud
Processing	Apache Spark	Batch & streaming compute	Databricks CE
Storage	Delta Lake	ACID-compliant data lake	Databricks CE
Orchestration	ADF / Airflow	Pipeline scheduling	Azure
Search	ChromaDB	Vector similarity search	Local / Databricks
Analytics	Spark SQL	KPI generation & reporting	Databricks CE

2. END-TO-END ARCHITECTURE DESIGN

2.1 Architecture Overview

The platform follows a Lakehouse architecture pattern that unifies data warehousing and data lake capabilities. It is designed as a hybrid system where batch and streaming pipelines coexist, analytical and real-time systems are integrated, and structured and unstructured data are both leveraged.

Architecture Flow:

Kafka (Confluent Cloud) → Bronze Layer (Delta) → Silver Layer (Delta) → Gold Layer (Delta)

Historical CSV → Bronze Layer (Delta) → Silver Layer (Delta) → Gold Layer (Delta)

Product Metadata → Embeddings → ChromaDB → Semantic Search

2.2 Architecture Components

Kafka Ingestion Layer

Confluent Cloud serves as the streaming message broker. A Python producer generates synthetic retail transactions and pushes them to the `smartgear_orders` Kafka topic. The topic is configured with multiple partitions to enable parallel processing and horizontal scalability.

Databricks Streaming + Batch Processing

Databricks Community Edition runs both Spark Structured Streaming jobs (consuming from Kafka in real-time) and batch jobs (ingesting historical CSV data). This unified compute platform eliminates the need for separate streaming and batch infrastructure.

Medallion Architecture

Data flows through three quality tiers — Bronze (raw), Silver (cleaned), and Gold (aggregated KPIs) — stored as Delta tables. Each layer adds progressively more business value while maintaining traceability back to raw sources.

Delta Lake

Delta Lake provides ACID transactions, schema enforcement, and time-travel capabilities over standard Parquet files. It acts as the reliable storage backbone for all three medallion layers.

Synapse Analytics

Azure Synapse Analytics provides dedicated SQL Pools for high-performance analytical queries on Gold layer data, enabling enterprise BI reporting with familiar SQL syntax.

Vector Search Layer

Product descriptions are embedded using the `all-MiniLM-L6-v2` sentence transformer model. The resulting vectors are stored in ChromaDB, enabling approximate nearest neighbour (ANN) based semantic search queries.

2.3 Why Lakehouse Over Traditional Data Warehouse?

Dimension	Lakehouse Advantage
Cost	Commodity cloud storage (S3/ADLS) is 10-100x cheaper than DW storage
Flexibility	Supports structured, semi-structured, and unstructured data natively
Streaming	Direct streaming ingestion without ETL staging overhead
ML/AI	Native support for ML workloads on the same data platform
Schema	Schema-on-read allows flexibility without rigid DDL constraints
Scalability	Decoupled compute and storage scale independently

2.4 Batch vs Streaming Trade-offs

- Batch processing: used for historical CSV ingestion, end-of-day KPI recalculation, and Synapse reporting. Lower cost, higher latency.
- Streaming processing: used for real-time Kafka ingestion and live dashboard updates. Lower latency, higher cost.
- Cost trade-offs arise from: continuous cluster uptime, Kafka partition count, Delta checkpoint storage, and streaming micro-batch frequency.

3. KAFKA PIPELINE & STREAMING

3.1 Kafka Theory (B1)

Topic, Partition, and Offset

Concept	Explanation
Topic	A named category/channel to which records are published. In this project: smartgear_orders
Partition	A topic is split into partitions for parallel processing. Each partition is an ordered, immutable log of messages.
Offset	A sequential ID assigned to each message within a partition. Consumers use offsets to track their position.

How Kafka Guarantees Durability

- Replication factor: each partition is replicated across N brokers
- Write-ahead logs: all messages are persisted to disk before acknowledging
- Leader-follower model: if a leader broker fails, a follower is promoted
- Retention policy: messages are kept for a configurable time period

Delivery Semantics

Semantic	Behaviour	Risk	Used When
At-most once	Message processed once or dropped	Data loss possible	Non-critical events
At-least once	Message may be reprocessed	Duplicates possible	General streaming
Exactly-once	Processed precisely once	Highest complexity	Financial / KPI pipelines

Consumer Group Rebalancing

When a consumer joins or leaves a consumer group, Kafka triggers a rebalancing event — reassigning partition ownership among active consumers. During rebalancing, consumption is temporarily paused, which can cause increased Kafka lag.

Why Partitioning Matters for Scalability

Partitions enable parallel consumption. With N partitions, up to N consumers in a group can read simultaneously. In this project, 3 partitions allow 3 parallel Databricks Spark tasks to consume from smartgear_orders, scaling throughput horizontally.

3.2 Synthetic Streaming Data Generator (B2)

A Python script generates realistic retail transaction events every few seconds and pushes them to Confluent Cloud Kafka:

```
import json, random, time
from datetime import datetime
from confluent_kafka import Producer

STORES = [f'STORE_{i}' for i in range(1, 11)]
PRODUCTS = ['Gaming Laptop', 'Wireless Headphones', 'Smartphone', 'Gaming Console', 'Smartwatch']
REGIONS = ['North', 'South', 'East', 'West']

def generate_event(order_id):
    return {
        "order_id": order_id,
        "timestamp": datetime.now().isoformat(),
        "store_id": random.choice(STORES),
        "product": random.choice(PRODUCTS),
        "quantity": random.randint(1, 10),
        "price": round(random.uniform(50, 2000), 2),
        "region": random.choice(REGIONS)
    }

producer = Producer({'bootstrap.servers': KAFKA_BOOTSTRAP, ...})
order_id = 1
while True:
    batch = [generate_event(order_id + i) for i in range(10)]
    for event in batch:
        producer.produce('smartgear_orders', json.dumps(event).encode())
    producer.flush()
    order_id += 10
    time.sleep(5)
```

3.3 Kafka Topic Configuration (B3)

Configuration	Value / Justification
Topic Name	smartgear_orders
Partitions	3 — allows 3 parallel Spark tasks; justifies regional parallel processing
Retention	7 days — balances storage cost vs. replayability for debugging
Security	SASL_SSL with API Key authentication via Confluent Cloud
Producer	Python confluent-kafka library with batch flush every 5 seconds
Consumer	Spark Structured Streaming with earliest offset reset

3.4 Spark Structured Streaming Integration (B4)

The following code establishes the Kafka-to-Bronze streaming pipeline with schema enforcement and checkpointing:

```
from pyspark.sql.types import *
from pyspark.sql.functions import *

# Schema definition
sales_schema = StructType([
    StructField('order_id', IntegerType(), True),
    StructField('timestamp', StringType(), True),
    StructField('store_id', IntegerType(), True),
    StructField('product', StringType(), True),
    StructField('quantity', IntegerType(), True),
    StructField('price', DoubleType(), True),
    StructField('region', StringType(), True)
])

# Read from Kafka
streaming_df = (spark.readStream.format('kafka')
    .option('kafka.bootstrap.servers', KAFKA_BOOTSTRAP)
    .option('subscribe', 'smartgear_orders')
    .option('kafka.security.protocol', 'SASL_SSL')
    .option('startingOffsets', 'earliest')
    .load())

# Parse JSON
parsed_df = (streaming_df
    .selectExpr('CAST(value AS STRING)')
    .select(from_json(col('value'), sales_schema).alias('data'))
    .select('data.*'))

# Write to Bronze (Delta) with checkpointing
bronze_query = (bronze_df.writeStream
    .format('delta')
    .outputMode('append')
    .option('checkpointLocation', '/Volumes/.../bronze_checkpoint_v1')
    .trigger(availableNow=True)
    .toTable('shopstream.default.bronze_kafka_sales'))
```


4. MEDALLION ARCHITECTURE — BRONZE / SILVER / GOLD

4.1 Bronze Layer — Raw Data Ingestion (C1)

Purpose: Store raw, unmodified events from Kafka as an append-only Delta table.

Table: shopstream.default.bronze_kafka_sales

Metadata Added: ingestion_time (current_timestamp), source ('kafka_stream')

The Bronze layer acts as the source-of-truth archive. It preserves original event fidelity, enables full replayability if downstream transformations fail, and provides the audit trail required for enterprise data governance.

```
bronze_df = (  
    parsed_df  
    .withColumn('ingestion_time', current_timestamp())  
    .withColumn('source', lit('kafka_stream'))  
)  
  
bronze_query = (bronze_df.writeStream  
    .format('delta')  
    .outputMode('append')  
    .option('checkpointLocation', '/Volumes/.../bronze_checkpoint_v1')  
    .trigger(availableNow=True)  
    .toTable('shopstream.default.bronze_kafka_sales'))
```

4.2 Silver Layer — Cleaning & Transformation (C2)

Purpose: Apply data quality rules and business transformations on Bronze data.

Table: shopstream.default.silver_sales

Input: shopstream.default.bronze_kafka_sales

Data Engineering Transformations

Transformation	Implementation & Justification
Null Handling	dropna() on order_id, product, price — these fields are essential for KPI accuracy. Imputing is not viable as defaults would corrupt revenue metrics.
Deduplication	dropDuplicates(['order_id', 'timestamp']) — composite key ensures unique event identification.
Region Standardization	initcap(col('region')) — normalizes casing for consistent GROUP BY aggregations.
Revenue Calculation	withColumn('revenue', col('quantity') * col('price')) — core business metric derived here.
Type Validation	Enforced via StructType schema at Bronze parse stage.

```
from pyspark.sql.functions import upper, initcap

silver_df = (
    bronze_df
    .dropna(subset=['order_id', 'product', 'price'])
    .dropDuplicates(['order_id', 'timestamp'])
    .withColumn('region', initcap(col('region')))
    .withColumn('revenue', col('quantity') * col('price'))
)

final_silver_df.write.mode('overwrite').saveAsTable(
    'shopstream.default.silver_sales'
)
```

Analytical Transformations

1. Daily Revenue Trend

```
daily_revenue = (
    silver_data
    .groupBy('timestamp')
    .agg(sum('Revenue').alias('daily_revenue'))
    .orderBy('timestamp')
)
```

2. Region-wise Contribution %

```
total_revenue =
silver_data.agg(sum('Revenue').alias('total')).collect()[0]['total']

region_contribution = (
    silver_data.groupBy('region')
    .agg(sum('Revenue').alias('region_revenue'))
)
```

```
.withColumn('contribution_percent',  
            round((col('region_revenue') / total_revenue) * 100, 2))  
)
```

3. Top 3 Stores per Region (Window Function)

```
from pyspark.sql.window import Window  
from pyspark.sql.functions import dense_rank  
  
window_spec = Window.partitionBy('region').orderBy(col('store_revenue').desc())  
  
top_stores = (  
    store_revenue  
    .withColumn('rank', dense_rank().over(window_spec))  
    .filter(col('rank') <= 3)  
)
```

4. Data Skew Identification

Regions with disproportionately high revenue (>30% of total) are flagged as skewed. This informs Spark partition strategy — skewed partitions cause executor imbalance and longer job durations.

5. GOLD LAYER KPIs & ADVANCED ANALYTICS

5.1 Business KPIs (C3)

Gold Layer Tables:

- shopstream.default.gold_total_revenue_kpi
- shopstream.default.gold_average_order_value
- shopstream.default.gold_top_products
- shopstream.default.gold_region_sales_kpi
- shopstream.default.gold_store_performance

Region-wise Total Revenue

```
%sql
SELECT
    region,
    SUM(Revenue) AS total_revenue
FROM silver_sales_view
GROUP BY region
ORDER BY total_revenue DESC;
```

Month-over-Month Growth

```
%sql
SELECT
    SUBSTRING(timestamp, 1, 7) AS month,
    SUM(Revenue) AS monthly_revenue
FROM silver_sales_view
GROUP BY month
ORDER BY month;
```

Average Order Value Per Region

```
%sql
SELECT
    region,
    AVG(Revenue) AS avg_order_value
FROM silver_sales_view
GROUP BY region
ORDER BY avg_order_value DESC;
```

Store Performance Segmentation

```
%sql
WITH store_revenue AS (
    SELECT store_id, SUM(Revenue) AS total_revenue
    FROM silver_sales_view GROUP BY store_id
)
SELECT
    store_id,
    total_revenue,
    CASE
        WHEN total_revenue >= 700 THEN 'High'
        WHEN total_revenue >= 400 THEN 'Medium'
        ELSE 'Low'
    END AS performance_segment
FROM store_revenue
ORDER BY total_revenue DESC;
```

5.2 Advanced SQL Analytics

Top 5 Products by Revenue (Global + Region-wise using DENSE_RANK)

```
-- Global ranking
SELECT product, SUM(Revenue) AS total_revenue,
       DENSE_RANK() OVER (ORDER BY SUM(Revenue) DESC) AS product_rank
FROM silver_sales_view
GROUP BY product ORDER BY product_rank;

-- Region-wise ranking
WITH regional_product_revenue AS (
    SELECT region, product, SUM(Revenue) AS total_revenue
    FROM silver_sales_view GROUP BY region, product
)
SELECT * FROM (
    SELECT region, product, total_revenue,
           DENSE_RANK() OVER (PARTITION BY region ORDER BY total_revenue DESC) AS
regional_rank
    FROM regional_product_revenue
) WHERE regional_rank <= 5;
```

Rolling Revenue — 7-Day Moving Average

```
WITH daily_revenue AS (  
    SELECT DATE(timestamp) AS order_date, SUM(Revenue) AS daily_revenue  
    FROM silver_sales_view GROUP BY DATE(timestamp)  
)  
SELECT  
    order_date,  
    daily_revenue,  
    AVG(daily_revenue) OVER (  
        ORDER BY order_date  
        ROWS BETWEEN 6 PRECEDING AND CURRENT ROW  
    ) AS rolling_7day_avg  
FROM daily_revenue ORDER BY order_date;
```

Anomaly Detection — Revenue Deviation > 30%

```
WITH daily_revenue AS (  
    SELECT DATE(timestamp) AS order_date, SUM(Revenue) AS daily_revenue  
    FROM silver_sales_view GROUP BY DATE(timestamp)  
) ,  
average_revenue AS (SELECT AVG(daily_revenue) AS avg_rev FROM daily_revenue)  
SELECT d.order_date, d.daily_revenue, a.avg_rev,  
    ROUND(ABS(d.daily_revenue - a.avg_rev) / a.avg_rev * 100, 2) AS  
deviation_percent  
FROM daily_revenue d CROSS JOIN average_revenue a  
WHERE ABS(d.daily_revenue - a.avg_rev) / a.avg_rev > 0.30  
ORDER BY deviation_percent DESC;
```

Customer Behavior Proxy (StoreID as Proxy)

Since customer IDs are unavailable in this dataset, StoreID is used as a proxy to simulate repeat transaction behavior patterns — a common technique in retail analytics.

```
SELECT  
    store_id,  
    COUNT(order_id) AS total_orders,  
    SUM(Revenue) AS total_revenue,  
    AVG(Revenue) AS avg_order_value  
FROM silver_sales_view  
GROUP BY store_id  
ORDER BY total_orders DESC;
```

6. SEMANTIC SEARCH & VECTOR DATABASE

6.1 Theory (F1)

What are Embeddings?

Embeddings are dense numerical vector representations of text, images, or other data. They capture semantic meaning — items with similar meaning produce vectors that are geometrically close in high-dimensional space. In this project, product descriptions are converted into 384-dimensional vectors using the all-MiniLM-L6-v2 Sentence Transformer model.

Why Cosine Similarity Works

Cosine similarity measures the angle between two vectors rather than their magnitude. Two product descriptions with semantically similar meaning produce vectors pointing in the same direction ($\cos \approx 1$), while unrelated descriptions produce perpendicular vectors ($\cos \approx 0$). This makes cosine similarity robust to sentence length variation.

Keyword Search vs Semantic Search

Dimension	Keyword Search vs Semantic Search
Mechanism	Keyword: exact string matching. Semantic: vector similarity in embedding space.
Synonyms	Keyword fails on synonyms. Semantic handles: 'phone' matches 'smartphone'.
Context	Keyword is context-blind. Semantic understands intent: 'gaming device' matches 'Gaming Console'.
Speed	Keyword: $O(n)$. Semantic with ANN: $O(\log n)$ with index.

What is ANN (Approximate Nearest Neighbour)?

ANN finds the most similar vectors without exhaustive comparison. Libraries like FAISS and ChromaDB build index structures (HNSW, IVF) that allow sub-linear search time over millions of vectors, enabling real-time semantic search at scale.

6.2 Practical Implementation (F2)

Product Metadata Dataset

Product ID	Product Name	Category	Description Summary
1	Gaming Laptop	Electronics	High performance laptop for gaming & graphics
2	Wireless Headphones	Audio	Noise cancelling with premium sound quality
3	Smartphone	Mobile	High-res camera, fast processor, Android
4	Gaming Console	Entertainment	Next-gen console with immersive graphics
5	Smartwatch	Wearable	Fitness tracking with heart rate monitoring

Embedding Generation & Vector Storage

```
from sentence_transformers import SentenceTransformer
import chromadb

model = SentenceTransformer('all-MiniLM-L6-v2')
embeddings = model.encode(products_df['description'].tolist())

client = chromadb.Client()
collection = client.create_collection(name='smartgear_products')

for idx, row in products_df.iterrows():
    collection.add(
        ids=[str(row['product_id'])],
        embeddings=[embeddings[idx].tolist()],
        documents=[row['description']],
        metadatas=[{'product_name': row['product_name'], 'category':
row['category']}]
    )
```

Semantic Search Function

```
def semantic_search(query_text, n_results=3):
    query_embedding = model.encode([query_text])
    results = collection.query(
        query_embeddings=query_embedding.tolist(),
        n_results=n_results
    )
    return results

# Example: query = 'high performance gaming device'
# Returns: Gaming Laptop, Gaming Console (highest cosine similarity)
```


7. ORCHESTRATION & PIPELINE DESIGN

7.1 Airflow Theory (D1)

DAG Structure

A DAG (Directed Acyclic Graph) represents task dependencies in Airflow. Tasks execute sequentially with no cyclic dependencies, ensuring deterministic pipeline execution. In this project: Kafka Ingestion → Bronze → Silver → Gold.

Idempotency

Pipeline idempotency means running the same pipeline multiple times produces identical results. Achieved here through Delta table overwrites, deduplication logic on order_id + timestamp, and Spark Structured Streaming checkpointing.

Backfills

Backfill reruns pipelines for historical time windows — for example, reprocessing missed Kafka batches into Bronze and Silver layers after a Confluent Cloud outage.

Retry Mechanisms

Airflow retries failed tasks automatically with configurable retry_count, retry_delay, and exponential backoff. This is critical for transient Databricks cluster startup failures.

Sensors

Airflow Sensors wait for external conditions before triggering downstream tasks. In this project, a Kafka topic availability sensor confirms smartgear_orders has active messages before launching the streaming job.

7.2 ADF Pipeline Design (D2)

Pipeline Stage	Details
Trigger	Step 1 (Kafka): Continuous/Event-based trigger. Steps 2-4: Tumbling window trigger every hour.
Step 1 — Kafka	Confluent Cloud producer generates events into smartgear_orders topic.
Step 2 — Bronze	Spark Structured Streaming writes append-only to bronze_kafka_sales Delta table.
Step 3 — Silver	Triggered after Bronze checkpoint. Applies cleaning, deduplication, revenue column.
Step 4 — Gold	Scheduled batch trigger. Generates KPI tables for executive dashboards.
Monitoring	Databricks monitoring + Azure Monitor + Kafka lag dashboards + ADF pipeline health.
Cost Strategy	Auto-scaling clusters, incremental Delta processing, scheduled cluster shutdown.

8. MONITORING & INCIDENT RESPONSE

8.1 Mini Case Study — Root Cause Analysis (Part G)

Observed Symptoms:

- Revenue mismatch between streaming and batch reports
- Sudden spike in Kafka consumer lag
- Missing records in Gold layer

Root Cause Analysis

Symptom	Possible Root Causes
Revenue Mismatch	Late-arriving Kafka events / duplicate processing / incorrect aggregation windows / schema inconsistencies between pipelines
Kafka Lag Spike	Slow Spark consumers / insufficient partitions / cluster resource exhaustion / large message bursts / network bottlenecks
Missing Gold Records	Failed Silver-to-Gold transformation / incorrect filter conditions / deduplication removing valid records / Delta write failures / incomplete checkpoint recovery

Debugging Strategy

- Validate Kafka pipeline — check topic health, partition distribution, consumer offsets, and compare produced vs consumed record counts using Confluent Cloud monitoring.
- Validate Bronze layer — verify raw event counts match Kafka offsets, inspect schema consistency, validate checkpoint recovery.
- Validate Silver transformations — audit null handling logic, duplicate removal rules, revenue calculation correctness.
- Validate Gold aggregations — verify SQL transformation correctness, window function outputs, reconcile with batch dataset totals.
- Resource monitoring — check Spark cluster CPU/memory, streaming micro-batch duration, executor failures, and shuffle bottlenecks.

8.2 Monitoring Framework

Layer	Metrics	Tools	Alert Threshold
Kafka	Consumer lag, throughput, partition imbalance	Confluent Cloud, Prometheus, Grafana	Lag > 10,000 msg/s
Streaming	Micro-batch time, failures, checkpoint health	Spark Streaming UI, Databricks	Batch time > 30s
Data Quality	Null %, duplicate %, schema drift	Great Expectations, Delta checks	Null % > 5%
Gold Layer	KPI freshness, missing partitions, revenue delta	Custom validation jobs, Azure Monitor	Freshness > 1hr

9. LEARNINGS & REFLECTIONS

9.1 Key Technical Learnings

- Lakehouse architecture successfully unifies batch and streaming pipelines on a single Delta Lake storage layer, eliminating the Lambda architecture's dual-code complexity.
- Spark Structured Streaming with checkpointing provides near-exactly-once processing semantics when combined with Delta's ACID transaction guarantees.
- Kafka partitioning strategy directly impacts parallel throughput — 3 partitions mapped to 3 Spark tasks provided observable speedup in streaming ingestion.
- Sentence Transformer embeddings captured semantic relationships that keyword search would miss entirely — 'gaming device' correctly surfaced both Gaming Laptop and Gaming Console.
- Delta time-travel proved invaluable for debugging — being able to query historical Bronze layer snapshots enabled root cause analysis of data quality issues.

9.2 Cost vs Performance Trade-offs (200-word Reflection)

The central cost vs performance tension in this project manifests most clearly in streaming infrastructure choices. Running Spark Structured Streaming on a continuously active Databricks cluster provides sub-minute data freshness but incurs constant DBU charges. A scheduled micro-batch approach (every 15 minutes) reduces cluster costs by roughly 90% while accepting slightly stale analytics — an acceptable trade-off for non-real-time dashboards.

Kafka retention is another cost lever: 7-day retention enables full pipeline replay for debugging but multiplies Confluent Cloud storage costs with data volume. Reducing retention to 24 hours would be appropriate once the pipeline reaches production stability.

Delta OPTIMIZE and ZORDER operations improve query performance on Gold layer tables by 3-5x but require additional compute time. Scheduling these during off-peak hours balances cost with analytical speed. Overall, a hybrid strategy — continuous streaming only for mission-critical real-time KPIs, batch for everything else — provides the optimal cost-to-latency ratio for a retail analytics platform of this scale.

9.3 Delta Lake Fundamentals (Q3)

Concept	Explanation
ACID Transactions	Delta adds a transaction log (<code>_delta_log</code>) over Parquet files, providing Atomicity (all-or-nothing writes), Consistency (schema enforcement), Isolation (concurrent reads/writes), and Durability (writes survive failures).
Schema Evolution	Delta supports additive schema changes (new columns) via MERGE schema option. Breaking changes are prevented, protecting downstream consumers.

Time Travel	Every Delta write creates a versioned snapshot. Query any historical version: <code>SELECT * FROM table VERSION AS OF 5</code> . Critical for data audits and debugging.
Streaming Pipelines	Delta is essential for streaming because it provides exactly-once write semantics via idempotent transactions, checkpointing integration, and the ability to read streaming + batch data from the same table.

9.4 Assumptions & Limitations

- Databricks Community Edition single-node clusters limit true parallel streaming; production would use multi-node clusters.
- Confluent Cloud free tier imposes message size and throughput limits — production would require a paid tier.
- Customer ID is absent from the dataset; StoreID is used as a behavioral proxy, which is an approximation.
- Vector search is implemented locally with ChromaDB; production would use a managed service like Pinecone or Azure Cognitive Search.

-----**END OF REPORT**-----